Leading Security Operations with Splunk (SIEM)

As a SOC Team Lead and Splunk Administrator in a Canada-based organization, my daily work is all about **turning log data into actionable insights**.

In a recent project with OpenSSH authentication logs, I:

- Built dedicated indexes and onboarded structured CSV logs
- Identified key fields: `src_ip`, `user`, `host`, `authentication`, `status`, and timestamps
- Created role-based access for Tier 1, Tier 2, Threat Intel, and Incident Response teams
- Developed searches, dashboards, and alerts to detect failed login spikes, brute-force attempts, and anomalous successes

Collaboration was just as important: mentoring analysts, refining alerts, enriching threat intelligence, and guiding incident response actions.

I also ensured **secure Splunk access** via HTTPS and proper role-based permissions.

What I've learned: **effective security is not just about tools — it's about processes, people, and smart use of technology.**

#CyberSecurity #SOCLeadership #Splunk #SIEM #BlueTeam #ThreatDetection #IncidentResponse #InfoSec

……..

Leading Security Operations with Splunk (SIEM)

My daily task as a Splunk Administrator and SOC Team Lead in a Canadian company is to transform log data into insights that can be put to use.

In a recent project with OpenSSH authentication logs, I:

1. Added structured CSV logs and created specialized indexes.
2. Identified key fields: `src_ip`, `user`, `host`, `authentication`, `status`, and timestamps
3. I established role-based access for the Incident Response, Threat Intel, and Tier 1 teams.

4. Created dashboards, searches, and alerts to identify abnormal successes, brute-force attempts, and failed login spikes.

Working together was equally crucial for guiding incident response efforts, improving threat intelligence, honing alerts, and mentoring analysts.

I also made sure that role-based permissions were set up correctly and that Splunk access was over HTTPS.

What I've discovered: procedure, people, and intelligent technology utilization are all important components of effective security.

#CyberSecurity #SOCLeadership #Splunk #SIEM #BlueTeam #ThreatDetection #IncidentResponse #InfoSec

……..

Living in the Logs: How I Track and Examine Traffic Anomalies Every Day Using Splunk SIEM

By Abimbola Fadipe - SOC Team Lead & Splunk Administrator, (Company Confidential)

07:45 AM Dashboards, and the First Login Spike

Each morning begins in the same manner. Splunk is already running and the laptop is open, a bottle of water beside me.

Splunk is the first system I log into each day as the SOC Team Lead and Splunk Administrator for a mid-sized Canadian technology business. Prior to emails and meetings, I would like to know one thing:

What took place during the night?

Our OpenSSH authentication monitoring pipeline, which is based on an openssh.csv log source, is one of my most practical projects and one that I myself constructed end-to-end. Our setup ingests logs from several sources. Brute-force attacks, credential stuffing, and compromised

accounts typically appear on SSH, which is still one of the most targeted services on the internet.

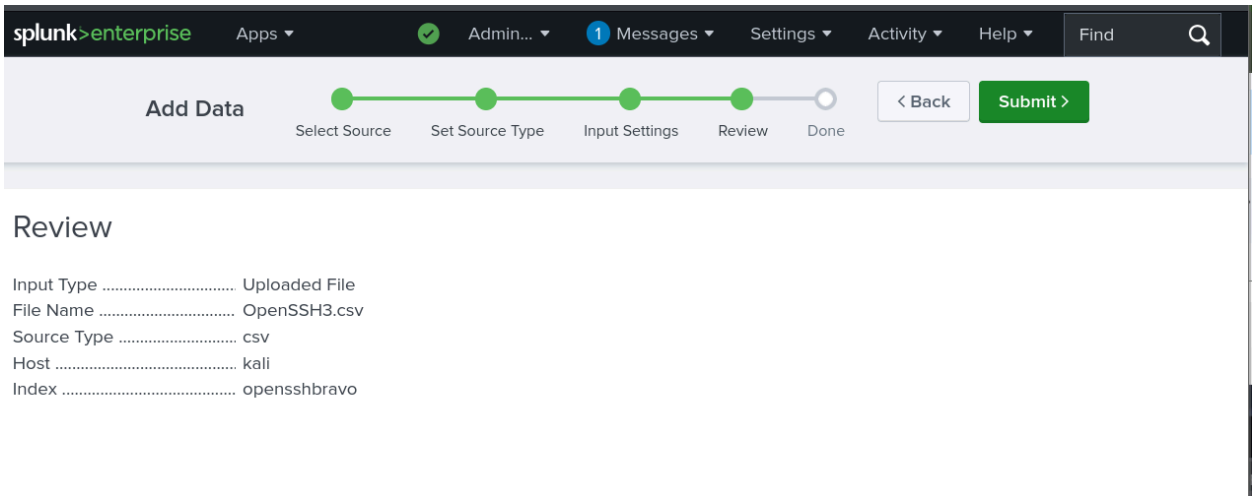
Splunk is our core nervous system, not just a tool.

Creating the OpenSSH Index: Foundation

Structure comes before analysis..

I did more than simply dumping the OpenSSH logs into a default index when I onboarded them. I made a special index:

index = opensshbravo



Why?

- SSH data isolation for quicker searching
- Role-based access control structured
- Simple alert tuning
- Less noise for Tier 1 - analysts

The index contains only SSH - authentication events, source IPs, successful logins, failed attempts, invalid users, hosts and timestamps.

The rules were made simple right from the very first day: that is, SSH authentication? then it belongs in the SSH index “opensshbravo”.

Users, Roles, and the Reality of SOC Access Control

Over-privileged access is one of the most common errors I observe in SOC systems. I built access to actual job functions rather than convenience in my capacity as Splunk administrator.

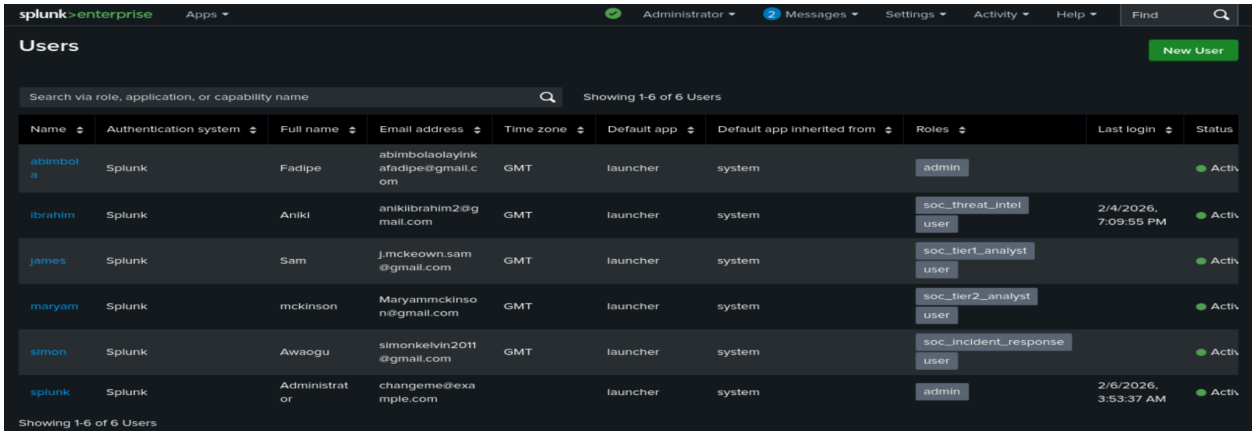
Splunk Users and Roles

I created distinct roles for:

- SOC Tier 1 Analyst
- SOC Tier 2 Analyst
- Threat Intelligence Analyst
- Incident Response Lead
- **Splunk Admin** (my role)

Each role had:

- Access that is either read-only or read-write as needed
- Access only pertinent indexes
- Permissions at the dashboard level
- No superfluous administrator rights



This was not a theoretical situation. When we began scaling, it made a difference.

Choosing the Fields That Actually Matter

It's simple to become overwhelmed with raw events when working with OpenSSH logs. In the beginning, I collaborated with the team to determine which fields were most operationally beneficial , rather than just those that were available.

For our `openssh.csv` project, we standardized on:

`_time` – when the event occurred

`src_ip` – source IP attempting access

`host` – target system

`user` – username used

`authentication` – SSH auth method

`status` – success / failed / invalid

`event` – raw event message

These fields became the backbone of:

- Searches
- Dashboards
- Alerts
- Incident timelines. Others were optional.

Daily Hunting: Searching the SSH Index

The actual work started when the data was organized and cleaned.

I perform a series of baseline searches each day in order to comprehend behavior:

```
index=opensshbravo
| stats count by src_ip
| sort - count
```

This instantly displays the most talked-about IPs , a straightforward yet effective method of locating brute-force sources.

Time-Binned Analysis : OpenSSH Failed Password Attempts

```
index=”opensshbravo” src_ip=”198.51.100.23” (”Failed password” OR ”Invalid user”)
| bin _time span=10m
| rex ”from (?<src_ip>\d+\.\d+\.\d+\.\d+)”
| rex ”for (invalid user )?(?<user>\S+)”
| eval status=”failed”
| stats count by _time user src_ip status
| sort - count
```

Successful Logins after multiple failures (a red flag):

```
Index ”opensshbravo” ("Failed password" OR ”Invalid user” OR ”Accepted password”)
| rex ”for (invalid user )?(?<user>\S+)”
| eval status=case(
   match(_raw,”Accepted password”), ”success”,
   match(_raw,”Failed password|Invalid user”), ”failed”
)
| sort 0 _time
| streamstats count(eval(status=”failed”)) as failed_count by src_ip user
| where status=”success” AND failed_count >= 5
| table _time src_ip user host failed_count status
```

These are queries I pose to the environment on a daily basis, not just searches.

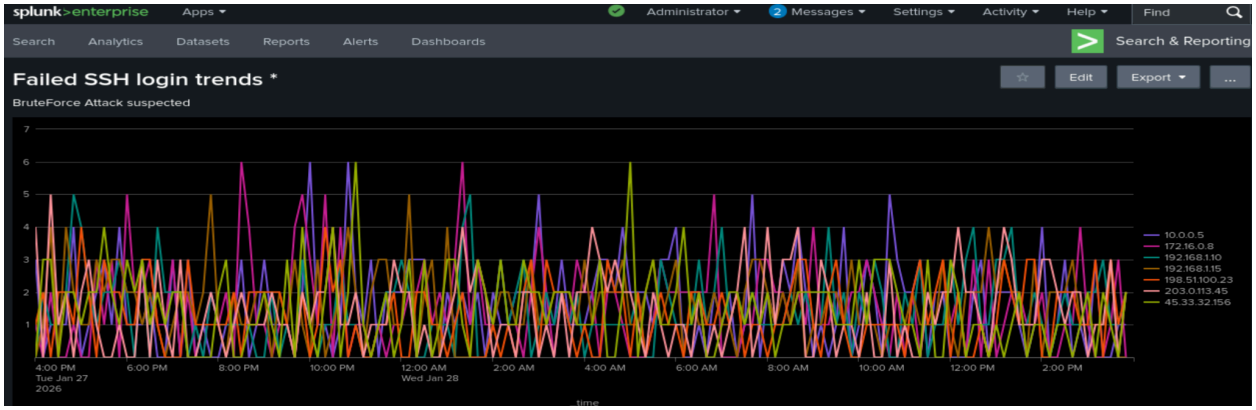
Searching to Dashboarding: Threats made Visible

Searches are awesome, but dashboards are toxic (lol).

I created **real-time shared SOC dashboards**, such as:

1. Trends of Failed SSH Logins

- Time-based perspective on unsuccessful endeavors
- Highlighted spikes
- Utilized by Tier 1 analysts while on shifts



2. Top Attacking IPs

- Ranked according to the number of failures
- Geographic enrichment using lookup tables
- This was used by the Threat Intelligence analysts

- Targeted accounts are highlighted?
- Which servers are vulnerable?

For analysts, dashboards were set to read-only. No unintentional changes. No panels are damaged.

Alerts: Signals were turned into Action

Dashboards show trends. Alerts demanded **response**.

I configured alerts such as:

- Several unsuccessful SSH logins from the same IP within ten minutes

- Successful logins after unsuccessful ones

- Attempts that are made repeatedly against nonexistent users

- Successful SSH access outside of regular business hours

Each alert included:

- Time
- Source IP
- Target user
- Host
- Event count
- Severity level

Alerts were routed to:

- SOC queues
- Emails
- Incident response workflows

Secure Access: HTTPS, Public IPs, and IAM

The security of Splunk access must not be compromised..

We enforced:

- **HTTPS for Splunk Web was enforced**

- Used a secure public IP

- Ports blocked by firewalls

- IAM based on roles in Splunk

No analyst used Splunk without:

- Having a position assigned to them
- Access authorization that is documented
- Visibility of audits

The weakest link should **never** be security tools!

Mentoring and Collaboration: Growing the Team

As Team Lead, I played a human role in addition to a technical one.

I collaborated extensively with analysts to:

- Describe the reasons for alerts' triggering, not merely their occurrence.
- Guide Tier 1 analysts through false positives.
- Coach correlation techniques to Tier 2 analysts.
- Assist Threat Intel analysts in mapping activity to **MITRE ATT&CK**

SOC Tier 1 Analysts

Focused on:

- Finding spikes in failed login attempts
- Verifying the alert's severity
- Increasingly suspicious trends

SOC Tier 2 Analysts

Handled:

- **Correlating or Linking events** related to authentication
- Analyzing / Examining the context of login
- Identifying accounts that have been compromised

Threat Intelligence Analysts

Responsible for:

- Enriching source IPs
- Mapping to MITRE ATT&CK techniques
- Intelligence sharing between teams

Incident Response Lead

Reviewed:

- Findings
- **Recommending containment actions**

- Documenting lessons learned in preparation for future incidents

Splunk became our **common language**.

Issues We Overcame During the Process

Not everything went as planned.

We dealt with:

- Role misconfigurations were preventing analysts from accessing Splunk.
- Remote access was blocked by firewall rules.
- Over-noisy alerts drowning real threats
- CSV field parsing inconsistencies

Each problem became a learning turn — and a documented fix.

Why This Matters

Monitoring OpenSSH logs isn’t just a breeze in the air.

However, the true impact is every compromise that is avoided, every early detection, and every analyst who gains confidence.

For me, Splunk is more than just dashboards and searches.

It’s:

- Awareness of the situation
- Team enablement
- Discipline in operations
- Continuous improvement

End of Day — Still Watching

Splunk continues to watch by the time I log off in the evening.

Dashboards are updated. Alerts are on watch. Shifts are rotated by analysts.

And I'll pose the same query once more in the morning, "What happened overnight?"

Because neither Splunk nor the logs in security ever sleep!